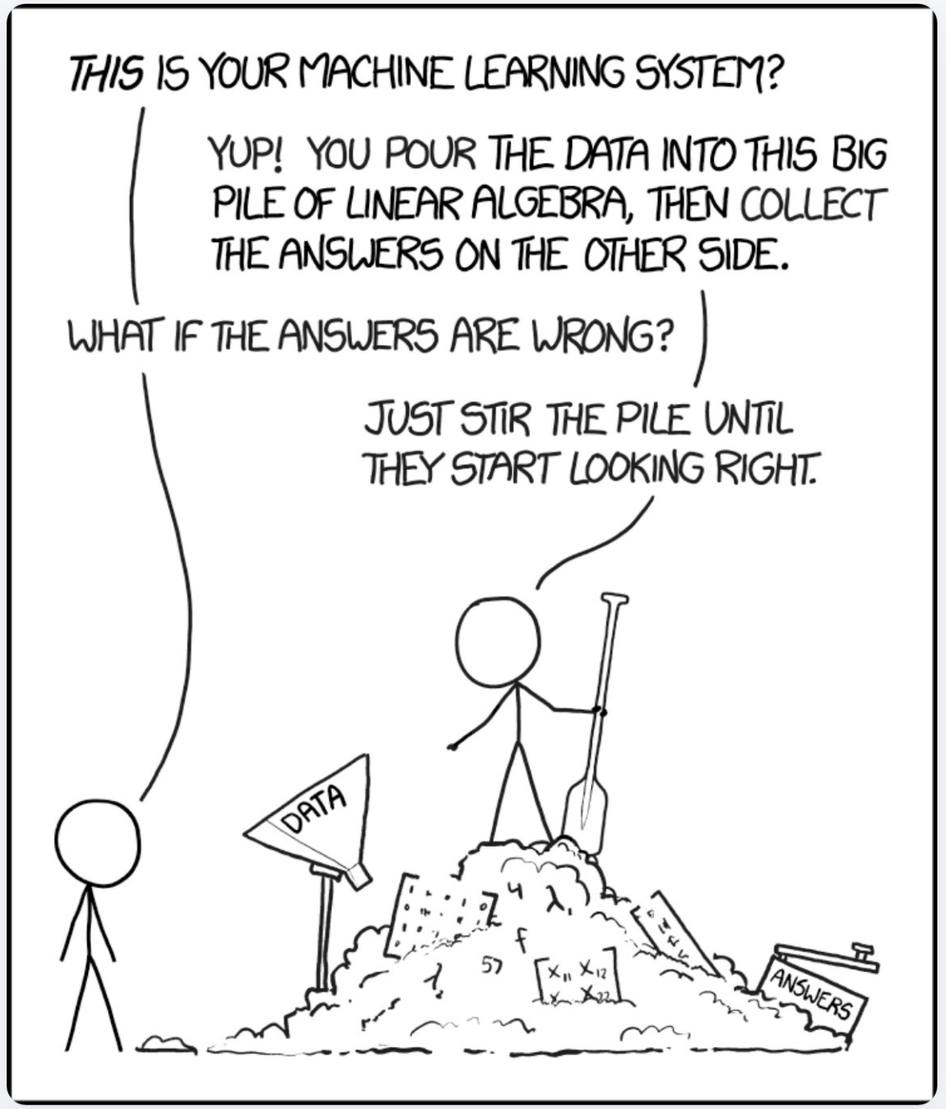


"Machine Learning", xkcd 1838, by Randall Munroe (xkcd.com/1838). CC BY-NC 2.5.

MACHINE LEARNING

Neural networks

Part 2: training a network.

LAST TIME

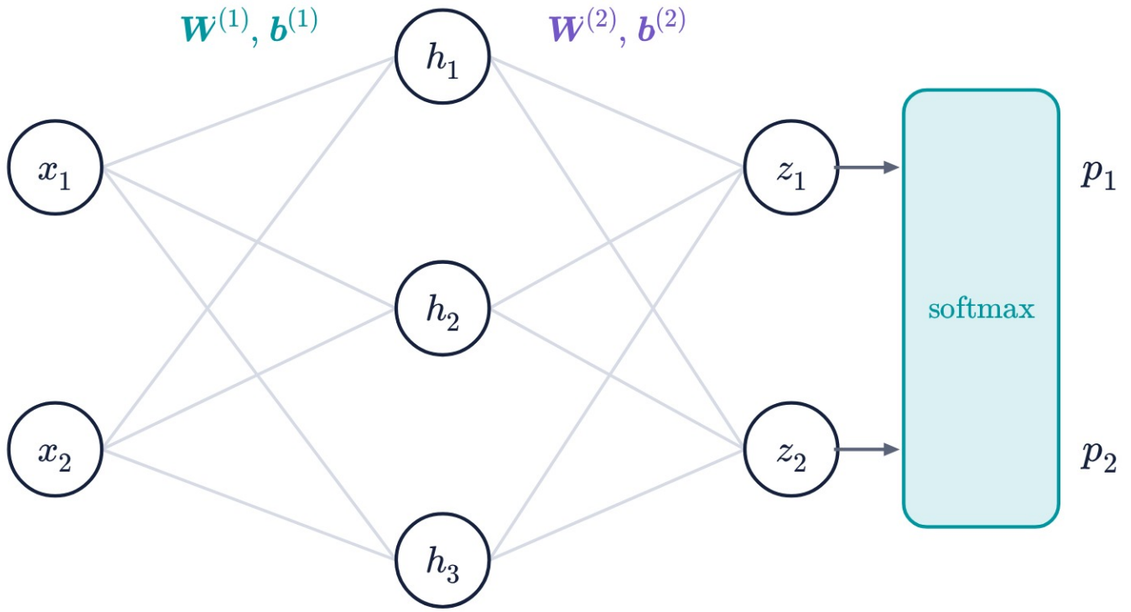
Forward propagation

$$\boldsymbol{h} = \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$$
$$\boldsymbol{z} = \boldsymbol{W}^{(2)}\boldsymbol{h} + \boldsymbol{b}^{(2)}$$
$$\boldsymbol{p} = \text{softmax}(\boldsymbol{z})$$

- Train by gradient descent:

  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$

- We need $\partial \mathcal{L} / \partial w$ for every weight w. Where does it come from?



BACKPROPAGATION

The network is one formula

A small network: two inputs, two hidden units, one output. Written out in full:

$$p = \sigma\Big(v_1\,\sigma(w_{11}x_1 + w_{12}x_2 + b_1) + v_2\,\sigma(w_{21}x_1 + w_{22}x_2 + b_2) + c\Big)$$

The loss of one training point, here squared error:

$$\mathcal{L} = \tfrac{1}{2}(p - y)^2$$

Nine weights, all inside one nested formula. We need $\partial\mathcal{L}/\partial w$ for each one.

A worked example

- One training point: $\boldsymbol{x} = (2, 1)$, label $y = 1$.
  The weights are on the connections.
  Sigmoid units; squared error $\mathcal{L} = \frac{1}{2}(p - y)^2$.
- Hidden layer:

  $a_1 = -0.5 \cdot 2 + 1 \cdot 1 + 1 = 1$

  $h_1 = \sigma(1) = 0.731$

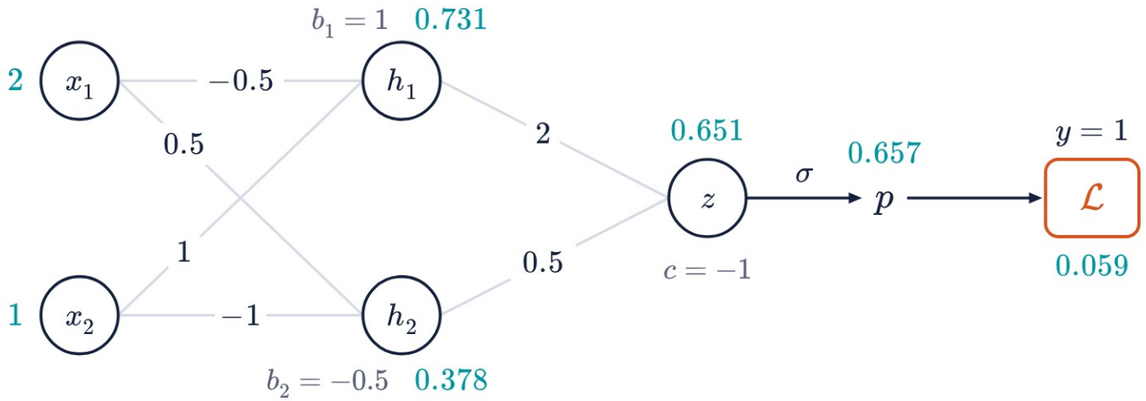
  $a_2 = 0.5 \cdot 2 - 1 \cdot 1 - 0.5 = -0.5$

  $h_2 = \sigma(-0.5) = 0.378$
- Output:

  $z = 2 \cdot 0.731 + 0.5 \cdot 0.378 - 1 = 0.651$

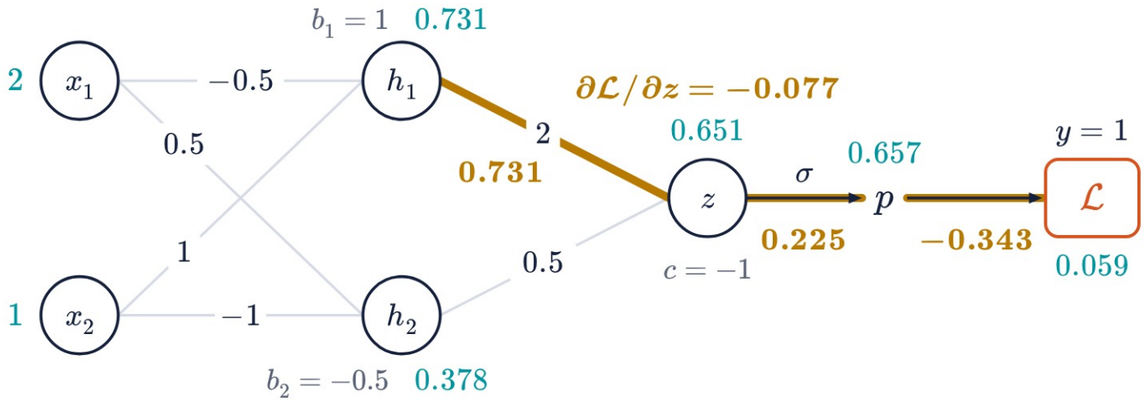
  $p = \sigma(0.651) = 0.657$
- Loss:

  $\mathcal{L} = \frac{1}{2}(0.657 - 1)^2 = 0.059$



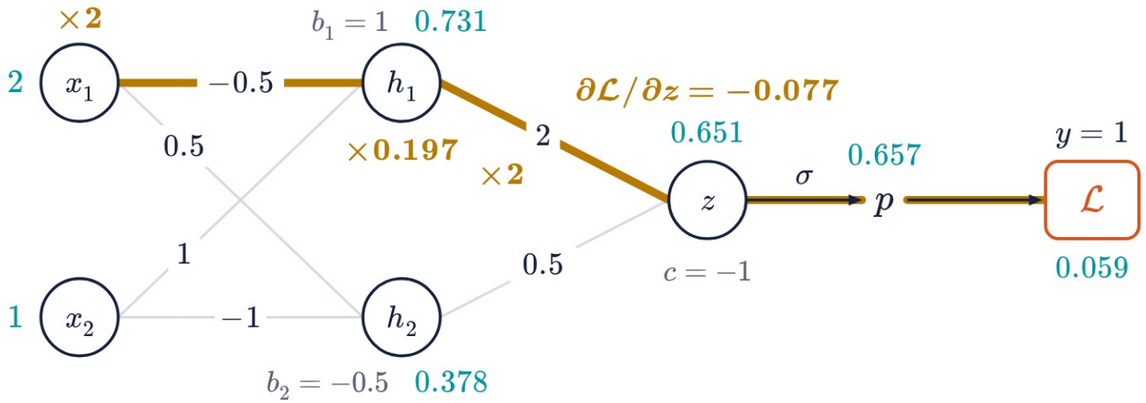
An output weight: v_1

- v_1 changes the loss along the highlighted path, $v_1 \to z \to p \to \mathcal{L}$. The chain rule multiplies the derivatives along it:
  $$\frac{\partial \mathcal{L}}{\partial v_1} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial v_1}$$
- $\partial \mathcal{L}/\partial p = p - y = 0.657 - 1 = -0.343$
- $\partial p/\partial z = p(1-p) = 0.225$
  0.657×0.343. The sigmoid's slope.
- $\partial z/\partial v_1 = h_1 = 0.731$
  Because $z = v_1 h_1 + v_2 h_2 + c$.
- Multiply. Keep the first two factors for later:
  $$\frac{\partial \mathcal{L}}{\partial z} = -0.343 \times 0.225 = -0.077$$
  $$\frac{\partial \mathcal{L}}{\partial v_1} = -0.077 \times 0.731 = -0.056$$



A hidden weight: w_{11}

- The path is longer:
  $w_{11} \to a_1 \to h_1 \to z \to p \to \mathcal{L}$.
  $$\frac{\partial \mathcal{L}}{\partial w_{11}} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_{11}}$$
- $\partial \mathcal{L}/\partial z = -0.077$, already computed for v_1. Reuse it.
- The new factors:
  $$\partial z/\partial h_1 = v_1 = 2$$
  $$\partial h_1/\partial a_1 = h_1(1-h_1) = 0.731 \times 0.269 = 0.197$$
  $$\partial a_1/\partial w_{11} = x_1 = 2$$
- Multiply:
  $$\frac{\partial \mathcal{L}}{\partial w_{11}} = -0.077 \times 2 \times 0.197 \times 2 = -0.061$$
  With another activation φ, use $\varphi'(a_1)$ in place of $h_1(1-h_1)$.



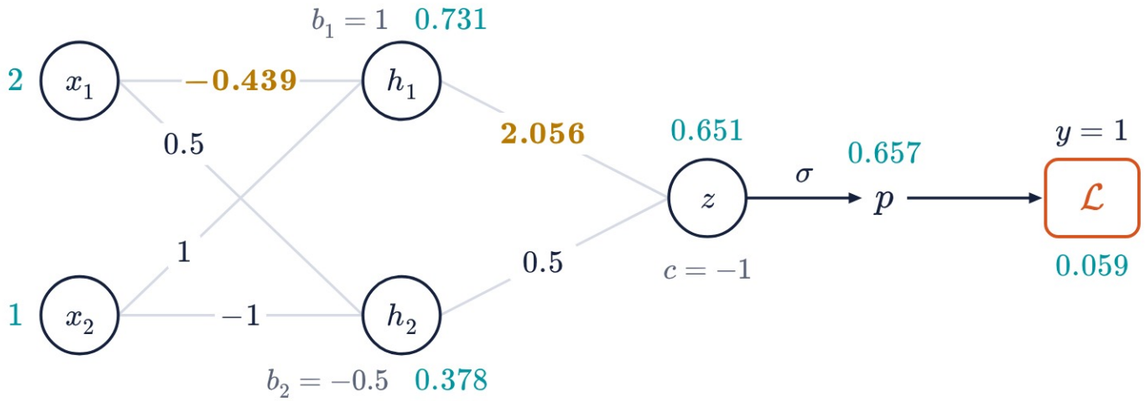
BACKPROPAGATION

Update the weights

- Gradient descent, with learning rate $\eta = 1$:

  $$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

- $v_1 \leftarrow 2 - (-0.056) = 2.056$
  $w_{11} \leftarrow -0.5 - (-0.061) = -0.439$
- Do the same for all nine weights, with every gradient computed from the old weights. Then the next training point.



BACKPROPAGATION

Now with cross-entropy

- Same network, same point, same forward pass: $p = 0.657$. Only the loss changes:

$$\begin{aligned}\mathcal{L} &= -y \log p - (1-y)\log(1-p) \\ &= -\log 0.657 = 0.420\end{aligned}$$

- So only the first factor changes:

$$\frac{\partial \mathcal{L}}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p} = -\frac{1}{0.657} = -1.522$$

- Times $\partial p / \partial z = p(1-p)$, it simplifies:

$$\frac{\partial \mathcal{L}}{\partial z} = -y(1-p) + (1-y)p = p - y = -0.343$$

- The rest of the chain is the same as before.

	chain	squared error	cross-entropy
$\partial \mathcal{L}/\partial z$		-0.077	-0.343
$\partial \mathcal{L}/\partial v_1$	$\partial \mathcal{L}/\partial z \times 0.731$	-0.056	-0.251
$\partial \mathcal{L}/\partial w_{11}$	$\partial \mathcal{L}/\partial z \times 2 \times 0.197 \times 2$	-0.061	-0.270

Two outputs

- Same input and first layer, so $h_1 = 0.731$, $h_2 = 0.378$. Now two outputs, a softmax and cross-entropy. The label is class 1: $\boldsymbol{y} = (1, 0)$.
  v_{kj} is the weight from h_j to z_k.
- The scores:

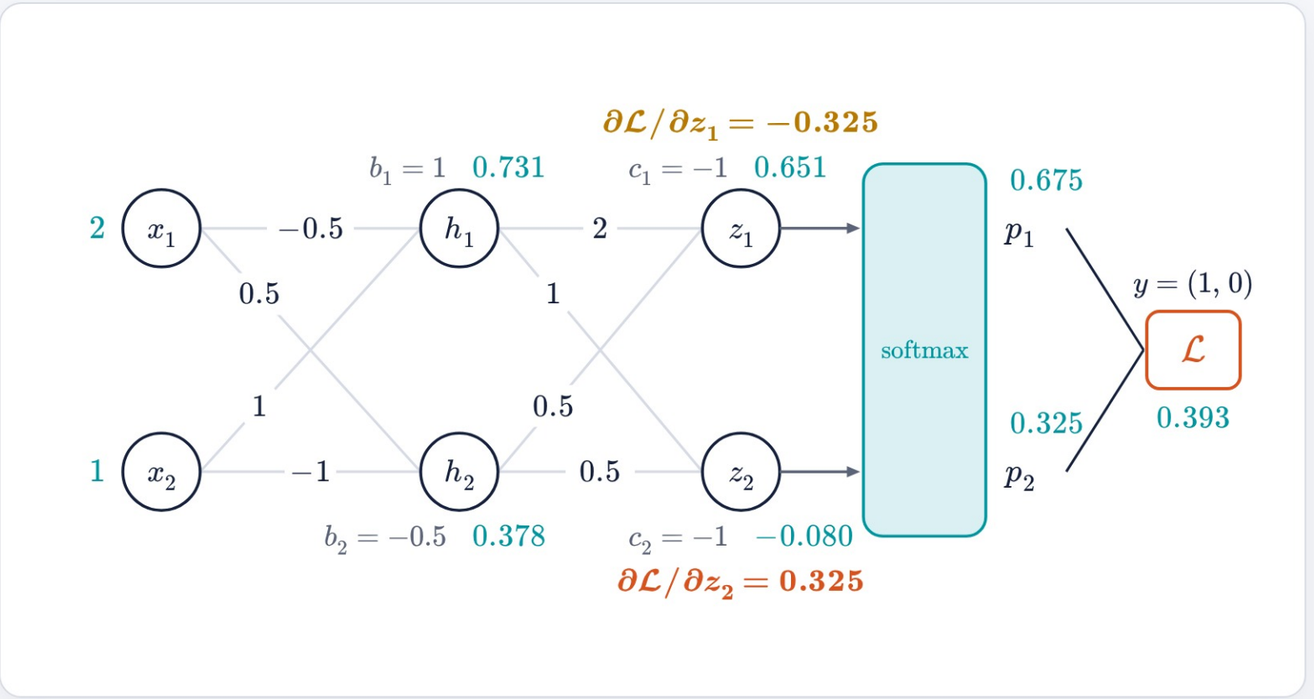
  $$z_1 = 2 \cdot 0.731 + 0.5 \cdot 0.378 - 1 = 0.651$$
  $$z_2 = 1 \cdot 0.731 + 0.5 \cdot 0.378 - 1 = -0.080$$

- The softmax and the loss:

  $$p_1 = \frac{e^{0.651}}{e^{0.651} + e^{-0.080}} = \frac{1.917}{1.917 + 0.923} = 0.675$$
  $$p_2 = 0.325, \qquad \mathcal{L} = -\log 0.675 = 0.393$$

- At each output, as before, $\partial \mathcal{L} / \partial z_k = p_k - y_k$:

  $$\frac{\partial \mathcal{L}}{\partial z_1} = 0.675 - 1 = -0.325, \quad \frac{\partial \mathcal{L}}{\partial z_2} = 0.325 - 0 = 0.325$$



Add the paths

- w_{11} changes h_1, and h_1 feeds both outputs. So w_{11} reaches the loss along two paths.
- Through z_1:

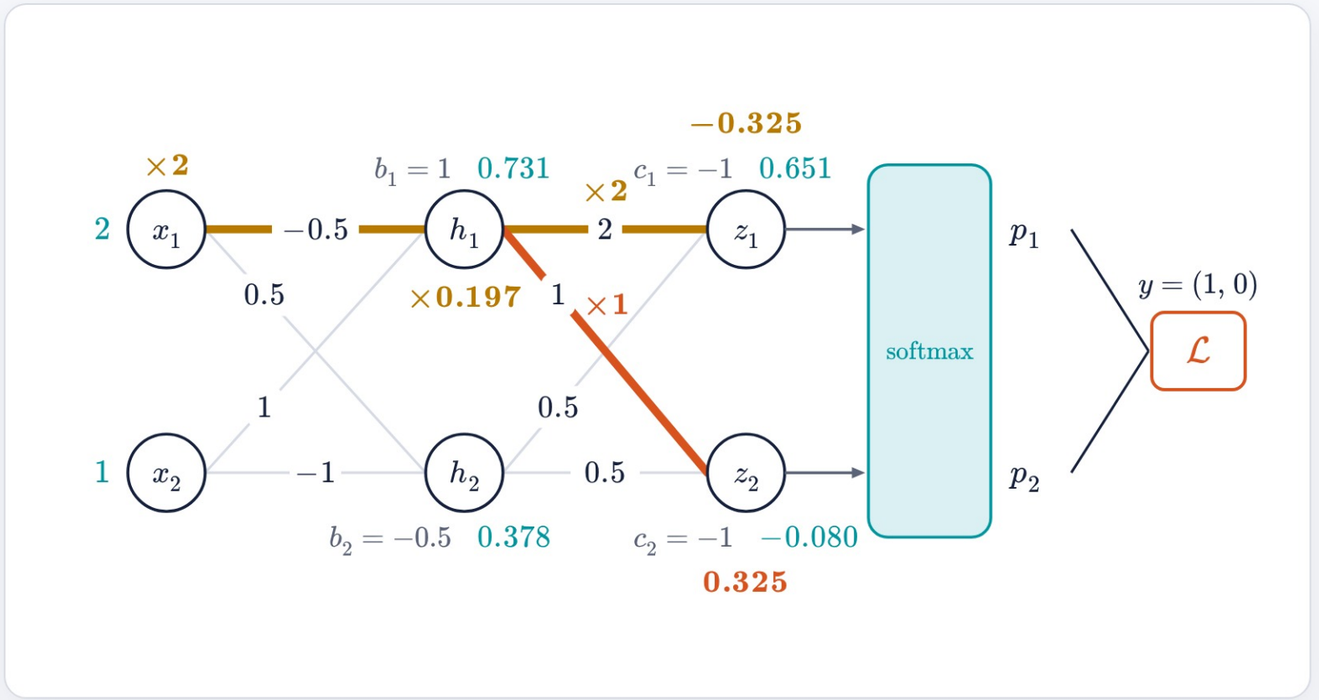
  $$(p_1 - y_1) \cdot v_{11} \cdot h_1(1 - h_1) \cdot x_1$$
  $$= -0.325 \times 2 \times 0.197 \times 2 = -0.256$$

- Through z_2:

  $$(p_2 - y_2) \cdot v_{21} \cdot h_1(1 - h_1) \cdot x_1$$
  $$= 0.325 \times 1 \times 0.197 \times 2 = 0.128$$

- Add them:

$$\frac{\partial \mathcal{L}}{\partial w_{11}} = \sum_k (p_k - y_k)\, v_{k1} \cdot h_1(1 - h_1) \cdot x_1$$
$$= -0.256 + 0.128 = -0.128$$

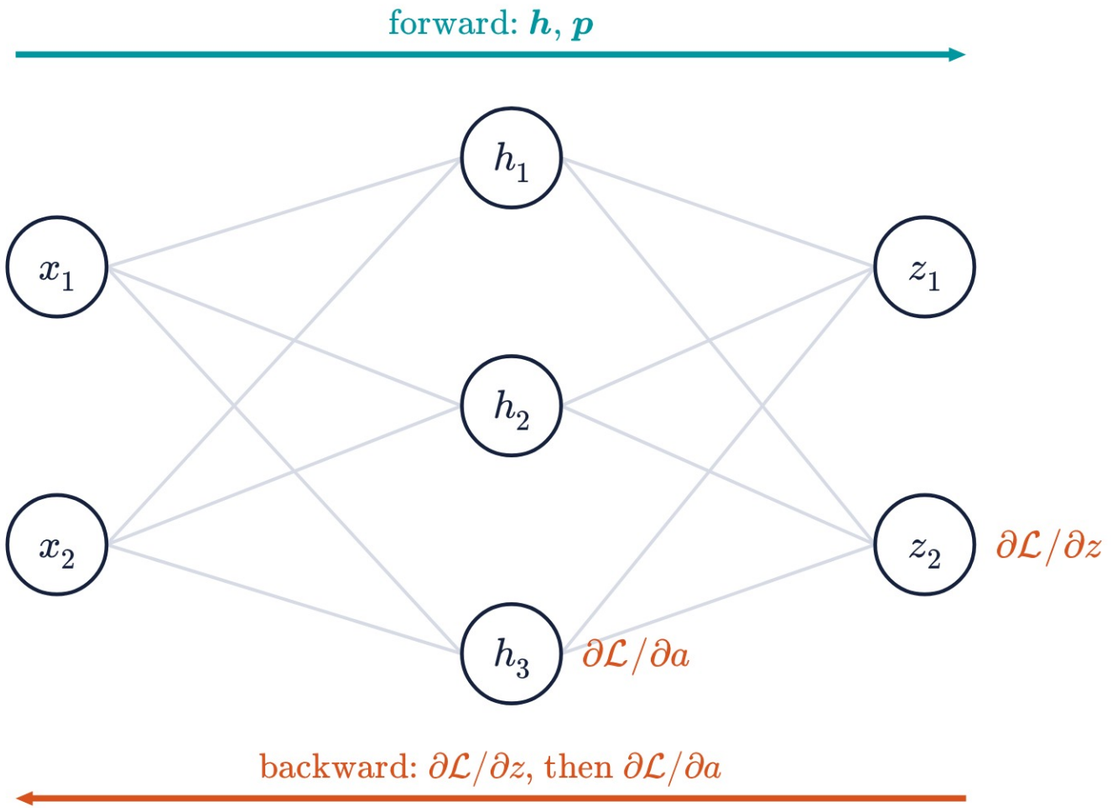


The recipe

1. **Forward pass**: compute and store $\boldsymbol{h}$ and $\boldsymbol{p}$.
2. **Error at each output**: $\partial \mathcal{L} / \partial z_k = p_k - y_k$.
3. **Backward pass**: each hidden unit adds up the errors of the outputs it feeds, then multiplies by its slope:

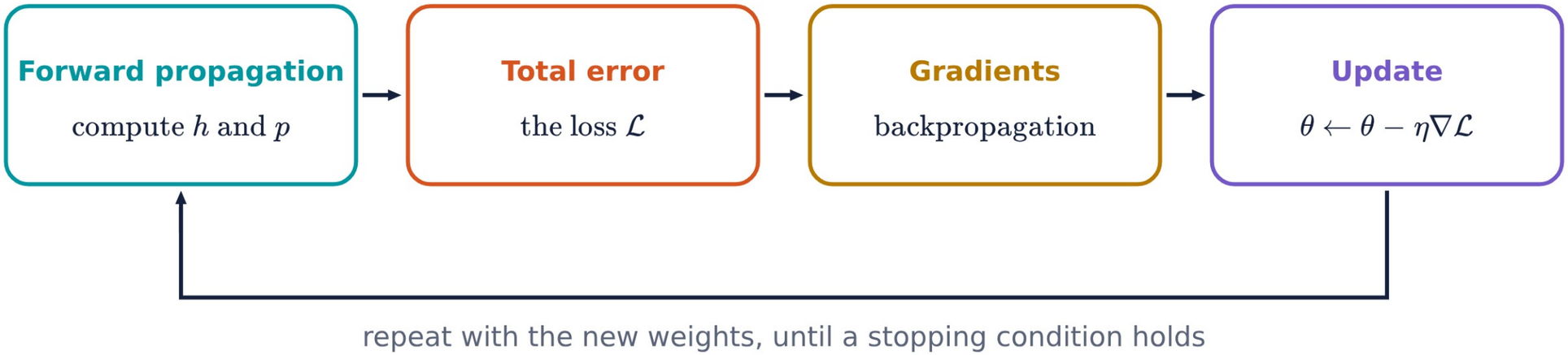
   $$\frac{\partial \mathcal{L}}{\partial a_j} = \left(\sum_k (p_k - y_k)\, v_{kj} \right) \cdot h_j (1 - h_j)$$

   With another activation, use $\varphi'(a_j)$. With more layers, repeat, layer by layer.
4. **Gradients**: each weight gets the error at its end times the value at its start: $(p_k - y_k)\, h_j$ for v_{kj}, $(\partial \mathcal{L} / \partial a_j)\, x_i$ for w_{ji}. A bias's start value is 1.
5. **Update** every weight, $w \leftarrow w - \eta\, \partial \mathcal{L} / \partial w$, using the old weights throughout.



PUTTING IT TOGETHER

The training loop



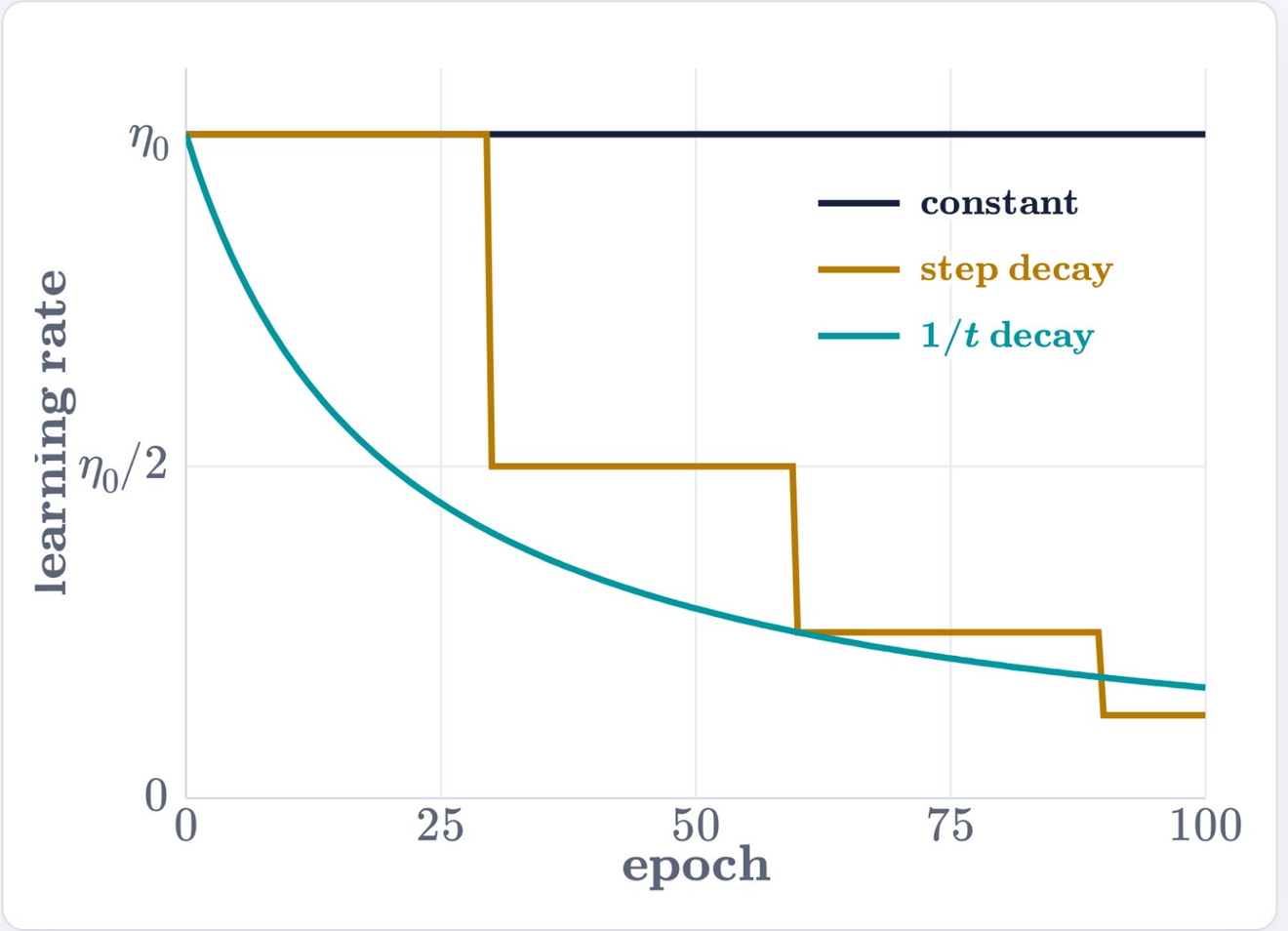
TRAINING IN PRACTICE

The learning rate

- The update:

  $$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$$

  Too small: slow. Too large: it overshoots. (Last class.)
- Large steps help early; small steps help near the end.
- A **learning-rate schedule** lowers η as training goes on.



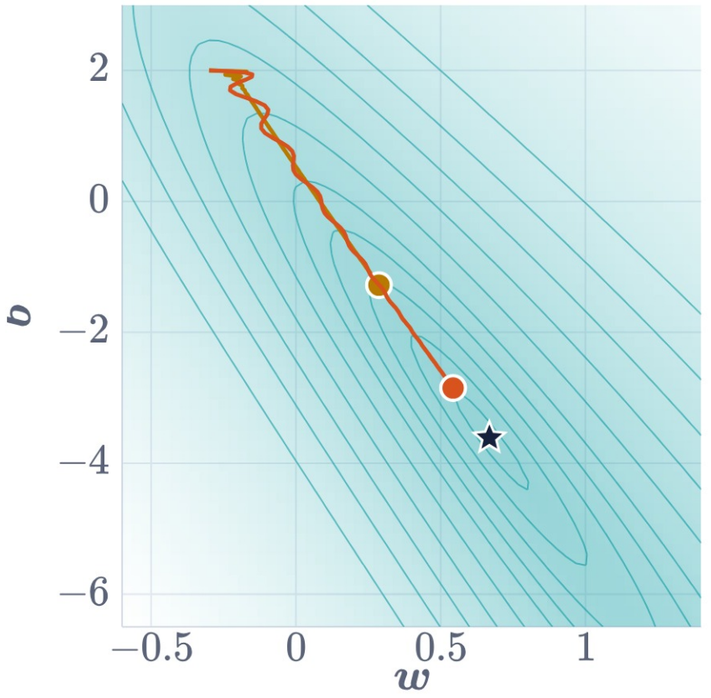
TRAINING IN PRACTICE

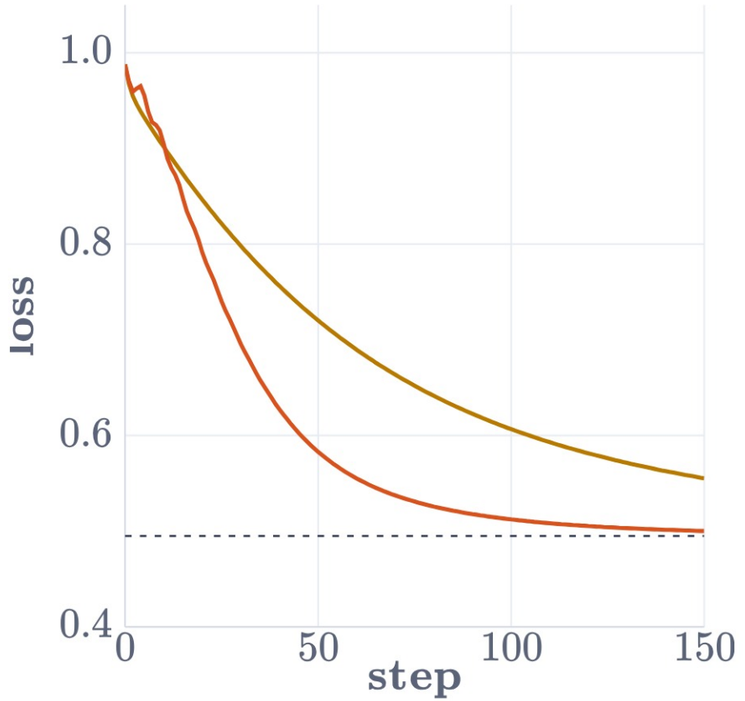
Momentum

- A long, narrow valley: plain gradient descent zigzags across it.
- Keep a running sum of past gradients:

  $$\boldsymbol{v} \leftarrow \beta \boldsymbol{v} + \nabla \mathcal{L}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \boldsymbol{v}$$

- The zigzags cancel; the steady downhill part adds up.





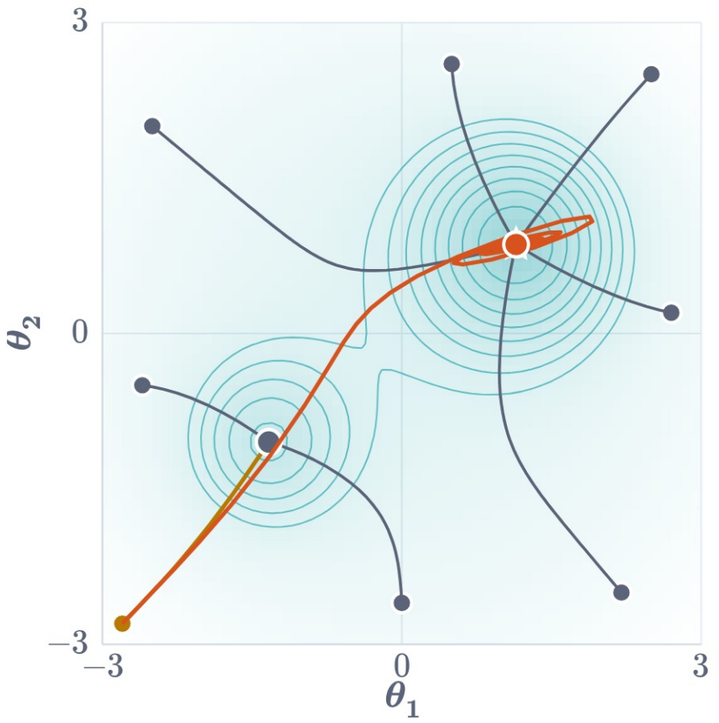
STEP	PLAIN LOSS	MOMENTUM LOSS
150	0.555	0.500

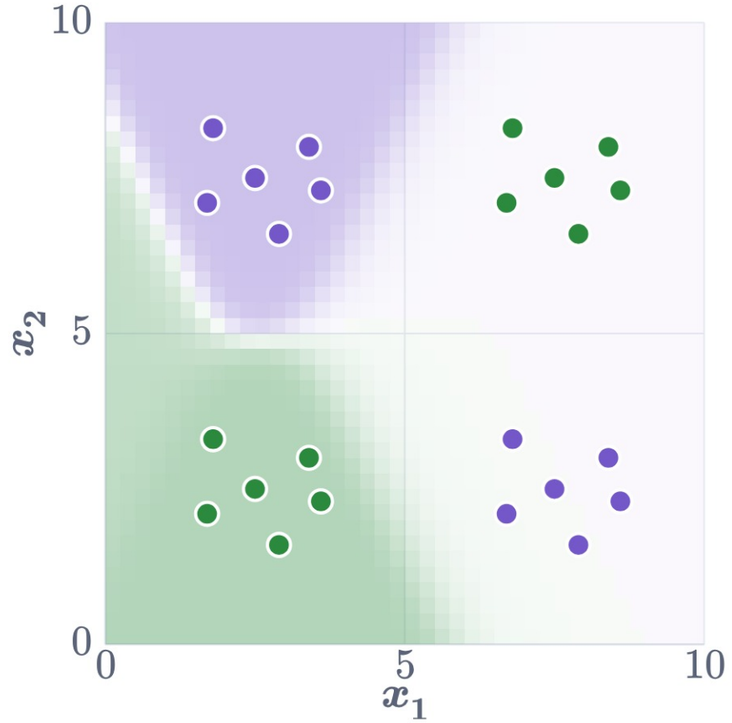
▶ step

TRAINING IN PRACTICE

Local minima

- Gradient descent stops in the valley it starts in.
- Restart from several starting points; keep the lowest loss.
- Momentum can roll through a shallow valley.
- It happens on real networks: the XOR network from a different start stalls.



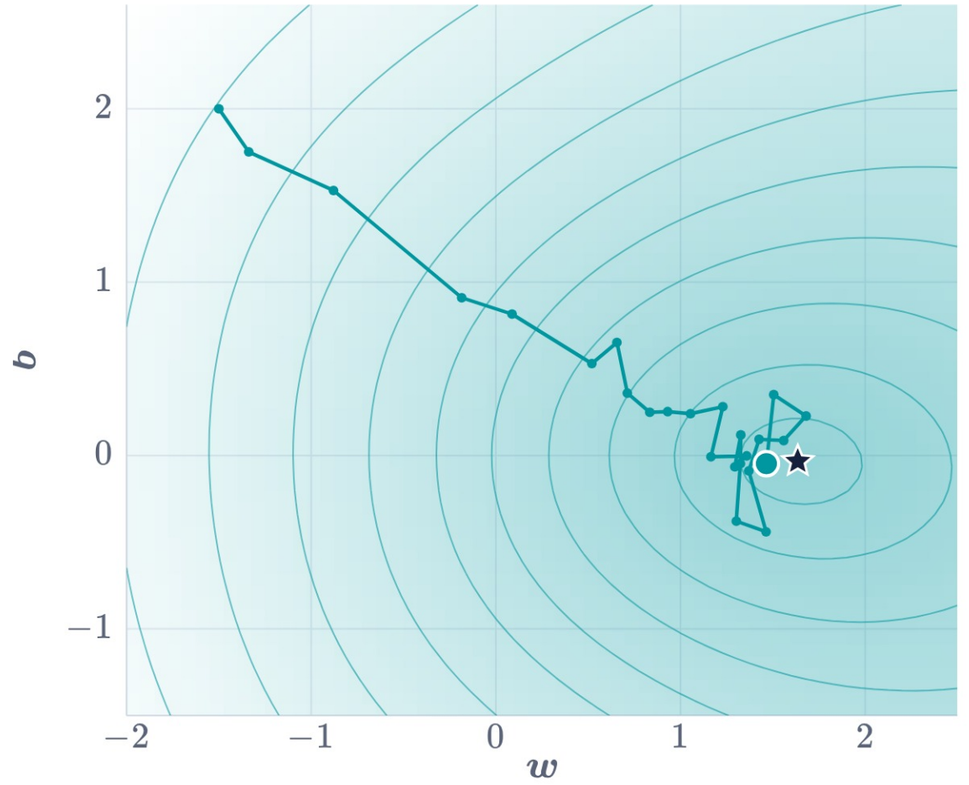


STEPS	LOSS	ACCURACY
4000	0.347	58%

TRAINING IN PRACTICE

How much data per step?

- **Batch**: the gradient from all the training points. One step per pass.
- **Stochastic**: from one point at a time. Many cheap, noisy steps.
- **Mini-batch**: from a few points, here 5. In between.
- An **epoch** is one pass through the training data.



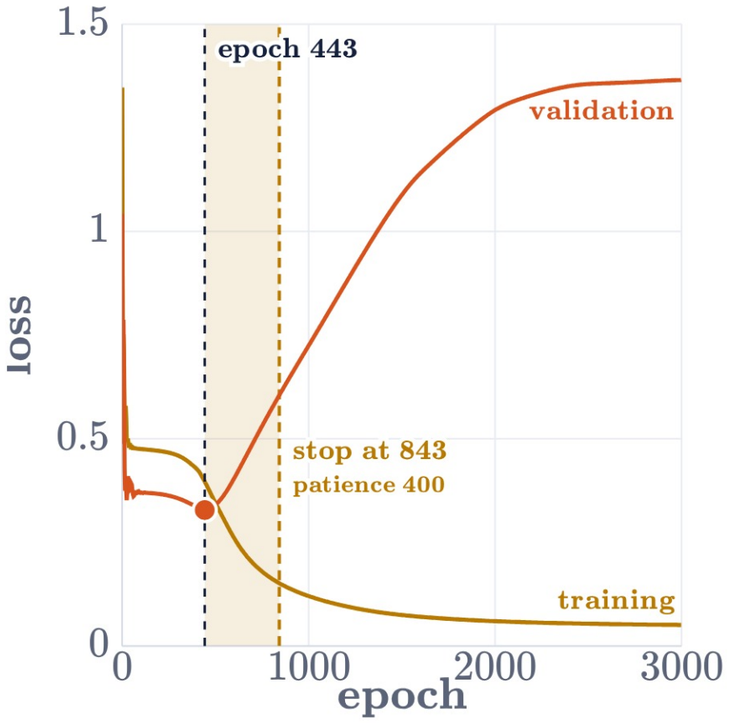
POINTS PER STEP	STEPS PER EPOCH	LOSS AFTER 6 EPOCHS
5	4	0.50

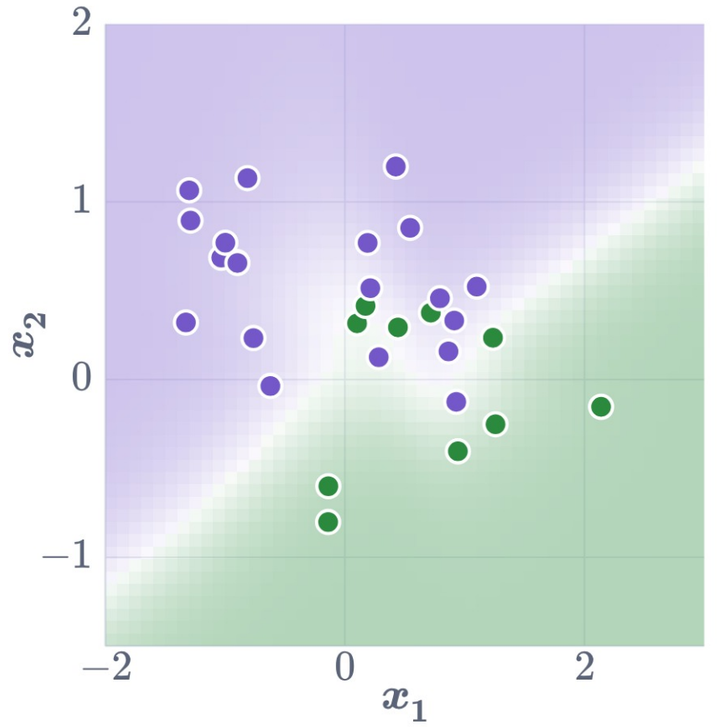
batch | stochastic | mini-batch | ▶

TRAINING IN PRACTICE

When to stop

- A maximum number of epochs; the weights stop changing; the loss is low enough.
- The training loss keeps going down.
- The validation loss goes down, then up: the network starts to overfit.
- **Early stopping**: keep the weights from the epoch with the lowest validation loss.
- **Patience**: stop once the validation loss has not improved for a set number of epochs.





EPOCH	TRAINING ACCURACY	VALIDATION ACCURACY
443	80%	89%

SUMMARY

Neural networks: characteristics

Strengths

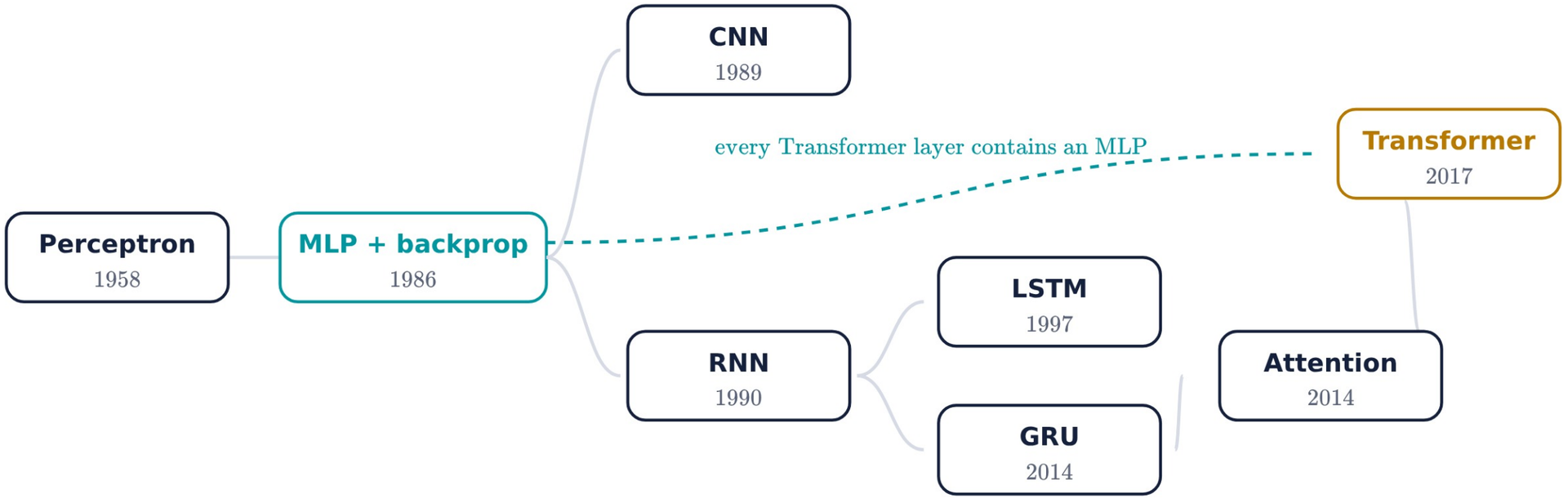
- Very flexible: one hidden layer with enough units can approximate any continuous function on a bounded region.
- They learn their own features, so less hand-made feature design.
- They improve with more data and computing power, and handle many outputs at once.
- Irrelevant inputs can be ignored: training can drive their weights to zero.

Weaknesses

- Non-convex training: the answer depends on the start, with no guarantee of the best weights.
- Many choices to tune: layers, units, η, optimizer, epochs.
- They need a lot of data, overfit small data sets, and are hard to interpret.
- Training is time-consuming: many passes over the data, and many runs to tune the choices above.

WHERE THIS GOES

The family tree



SUMMARY

Takeaways

Backpropagation

The chain rule, from the output back. At the output, $p_k - y_k$; a hidden unit adds up the paths through every output it feeds. Each weight's gradient is the error at its end times the value at its start.

Optimizers

Learning-rate schedules and momentum.

Batches

Batch, mini-batch, stochastic. An epoch is one pass through the data.

Stopping

Watch the validation loss; stop early.

Thank you.

Questions?